COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	9/22/2026	**DATE SUBMITTED:**	9/22/2026
NAME:	Payas, John Paul S.	**INSTRUCTOR:**	Sir. Jimlord Quejado

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

ILO A:
Task 1:

main.cpp

```cpp
#include <iostream>
#include "tree.h"

int main(){

    Tree<char>* A = new Tree<char>('A');
    Tree<char>* B = new Tree<char>('B');
    Tree<char>* C = new Tree<char>('C');
    Tree<char>* D = new Tree<char>('D');
    Tree<char>* E = new Tree<char>('E');
    Tree<char>* F = new Tree<char>('F');
    Tree<char>* G = new Tree<char>('G');

    Tree<char>* H = new Tree<char>('H');
    Tree<char>* I = new Tree<char>('I');
    Tree<char>* J = new Tree<char>('J');
    Tree<char>* K = new Tree<char>('K');
    Tree<char>* L = new Tree<char>('L');
    Tree<char>* M = new Tree<char>('M');
    Tree<char>* N = new Tree<char>('N');

    Tree<char>* P = new Tree<char>('P');
    Tree<char>* Q = new Tree<char>('Q');

    A->insertChild(B);
    A->insertChild(C);
    A->insertChild(D);
    A->insertChild(E);
    A->insertChild(F);
    A->insertChild(G);

    D->insertChild(H);

    E->insertChild(I);
    E->insertChild(J);

    J->insertChild(P);
    J->insertChild(Q);

    F->insertChild(K);
    F->insertChild(L);
    F->insertChild(M);

    G->insertChild(N);

    return 0;
}
```

tree.h

```h
#ifndef TREE_H
#define TREE_H

#include <iostream>

template<typename T>
class Tree{
    private:
    T key;
    Tree<T>* child;
    Tree<T>* sibling;

    public:
    Tree(T value);
    void insertChild(Tree<T>* node);
    void preOrder();
    void inOrder();
    void postOrder();
    bool findData(T value);
};

template<typename T>
Tree<T>::Tree(T value){
    key = value;
    child = nullptr;
    sibling = nullptr;
}

template<typename T>
void Tree<T>::insertChild(Tree<T>* node){

    if(child == nullptr){
        child = node;
    }
    else{
        Tree<T>* temp = child;

        while(temp->sibling != nullptr){
            temp = temp->sibling;
        }

        temp->sibling = node;
    }
}

template<typename T>
void Tree<T>::preOrder(){
```

```
    if(child != nullptr){
        child->preOrder();
    }

    if(sibling != nullptr){
        sibling->preOrder();
    }
}

template<typename T>
void Tree<T>::inOrder(){

    if(child != nullptr){
        child->inOrder();
    }

    std::cout << key << " ";

    if(sibling != nullptr){
        sibling->inOrder();
    }
}

template<typename T>
void Tree<T>::postOrder(){

    if(child != nullptr){
        child->postOrder();
    }

    if(sibling != nullptr){
        sibling->postOrder();
    }

    std::cout << key << " ";
}

template<typename T>
bool Tree<T>::findData(T value){

    if(key == value){
        return true;
    }

    if(child != nullptr){
        if(child->findData(value)){
            return true;
```

```
            return true;
        }
    }

    if(sibling != nullptr){
        if(sibling->findData(value)){
            return true;
        }
    }

    return false;
}

#endif
```

Task 2:

Node:	Height:	Depth:
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2

N	0	2
P	0	3
Q	0	3

ILO B:

Task 3:

3.1.

Pre-order	A B C D H E I J P Q F K L M G N
Post-order	B C H D I P Q J E K L M F N G A
In-order	B A C D H E I J P Q F K L M G N

3.2 .

Pre-order	`Pre-order output: A B C D H E I J P Q F K L M G N`
	
	`Find J using Pre-order: J was found!`
	
	Observation:
	
	The output of the function is the same as the pre-order result obtained by hand in #1.

Post-order	`In-order output: B C H D I P Q J E K L M F N G A`
	
	`Find J using In-order: J was found!`
	
	Observation:
	
	The output of the function is the same as the post-order result obtained by hand in #1.

In-order	`Post-order output: H Q P J I M L K N G F E D C B A`
	
	`Find J using Post-order: J was found!`
	
	Observation:

	The function follows the same child/sibling traversal used in the code, but in-order traversal for a general tree depends on the convention used.

Code:

```
    std::cout << "Find J using Pre-order: ";
    A->findData(1, 'J');
    std::cout << std::endl;

    std::cout << "Find J using In-order: ";
    A->findData(2, 'J');
    std::cout << std::endl;

    std::cout << "Find J using Post-order: ";
    A->findData(3, 'J');
    std::cout << std::endl;

    std::cout << "Find O using Pre-order: ";
    A->findData(1, 'O');
    std::cout << std::endl;

    std::cout << "Find O using In-order: ";
    A->findData(2, 'O');
    std::cout << std::endl;

    std::cout << "Find O using Post-order: ";
    A->findData(3, 'O');
    std::cout << std::endl;

    return 0;
}
```

Output:

```
Pre-order output: A B C D H E I J P Q F K L M G N
In-order output: B C H D I P Q J E K L M F N G A
Post-order output: H Q P J I M L K N G F E D C B A
```

```
Find J using Pre-order: J was found!
Find J using In-order: J was found!
Find J using Post-order: J was found!
Find O using Pre-order: O was found!
Find O using In-order: O was found!
Find O using Post-order: O was found!
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1:

Code:	
main.cpp	

```cpp
#include <iostream>
#include "BST.h"

int main(){

    BinarySearchTree<int>* tree = new BinarySearchTree(2);

    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);

    std::cout<<"Pre-order output: ";
    tree->preOrder();
    std::cout<<std::endl;

    std::cout<<"in-order output: ";
    tree->inOrder();
    std::cout<<std::endl;

    std::cout<<"Post-order output: ";
    tree->postOrder();
    std::cout<<std::endl;



    return 0;
}
```

BST.h

```h
#ifndef BST_H
#define BST_H

template<typename T>
class BinarySearchTree{
    private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    public:
    BinarySearchTree(T value);
    void insert(T value);
    void preOrder();
    void inOrder();
    void postOrder();

};

template<typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}

template<typename T>
void BinarySearchTree<T>::insert(T value){

    if(value < key ){

        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }else{
            left->insert(value);
        }
    }
    else if(value > key){
        if (right == nullptr){
            right = new BinarySearchTree<T>(value);
        }else{
            right ->insert(value);
        }
    }
}

template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<< key << " ";
```

```
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}

template<typename T>
void BinarySearchTree<T>::inOrder(){

    if(left != nullptr){
        left->inOrder();
    }
    std::cout<< key << " ";

    if(right != nullptr){
        right->inOrder();
    }
}

template<typename T>
void BinarySearchTree<T>::postOrder(){

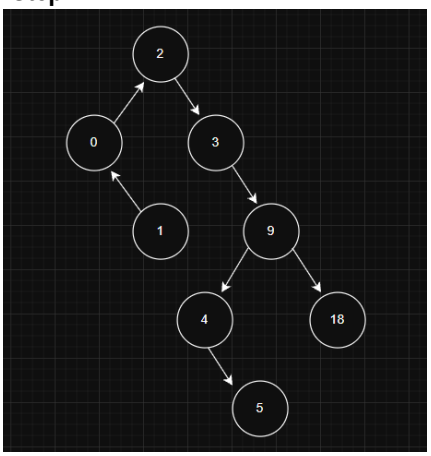
    if(left != nullptr){
        left->postOrder();
    }

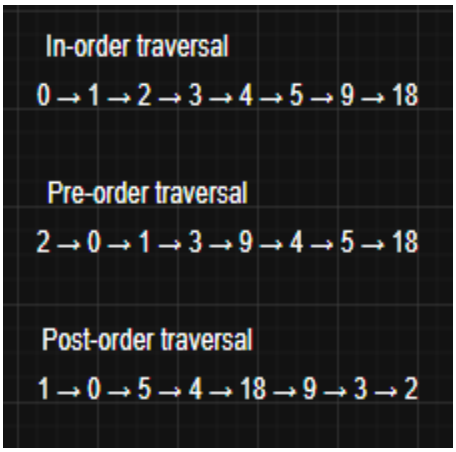
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<< key << " ";
}



#endif
```

Step 2:





Step 3:

```
in-order output: 0 1 2 3 4 5 9 18
```

The in-order traversal was performed using the inOrder() function

Pre-order Traversal

Code:

```
template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<< key << " ";

    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

Output:

```
Pre-order output: 2 0 1 3 9 4 5 18
```

Observation:

The output is the same as the pre-order traversal indicated in the diagram in item #2

Post-order Traversal

Code:

```
template<typename T>
void BinarySearchTree<T>::postOrder(){

    if(left != nullptr){
        left->postOrder();
    }

    if(right != nullptr){
        right->postOrder();
    }
    std::cout<< key << " ";
}
```

Output:

```
Post-order output: 1 0 5 4 18 9 3 2
```

Observation

The output is the same as the post-order traversal indicated in the diagram in item #2

Summary of Lessons Learned

I learned how to create and represent a tree using linked lists. I also learned how pre-order, in-order, and post-order traversal methods work and how their outputs differ. I learned how the findData() function can be used to search for a specific value in a tree.

Analysis of the Procedure

The procedure nodes are connected using child and sibling pointers. Creating the tree first and then applying the different traversal functions made it easier to understand how each node is visited. Comparing the manual results with the program output also helped verify that the traversal functions were working correctly.

Analysis of the Supplementary Activity

The Binary Search Trees and how values are placed based on whether they are smaller or larger than the current node. I also learned that in-order traversal of a Binary Search Tree produces the values in ascending order, while pre-order and post-order produce different sequences based on their traversal rules.

Concluding Statement / Feedback

I still need to improve my understanding of tree traversal, especially in-order traversal for general trees, and become more confident in implementing tree functions without errors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chat gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It fixed my grammar but it was my idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.